

Fundamentos de programación Python para el análisis de datos

Descripción	Este módulo proporciona una introducción a la programación utilizando Python, enfocándose en desarrollar las habilidades fundamentales necesarias para codificar rutinas que resuelvan problemas de baja complejidad. El módulo está diseñado para construir una base sólida desde los conceptos básicos hasta la programación orientada a objetos.
Competencia general	Codificar rutinas para dar solución a una problemática de baja complejidad utilizando el lenguaje Python.

Unidad	Aprendizaje esperado	Criterios de evaluación	Contenidos	Duración
El lenguaje python	Aplicar las principales herramientas del lenguaje Python para resolver distintas problemáticas en el entorno de trabajo.	- Reconoce las aplicaciones en donde puede ser utilizado el lenguaje Python. - Aplica las principales herramientas de python características en el entorno de trabajo de visual studio code.	- Reseña del lenguaje python. - Propósito del lenguaje python. - Principales características del lenguaje. - Versiones de python. - Entorno de trabajo y herramientas: - Visual Studio Code. - El entorno anaconda. - Jupyter notebooks. - El editor spyder. - Google collab.	2 horas
Sentencias básicas del lenguaje python	Aplicar el concepto de variable, tipos de datos fundamentales y expresiones aritméticas utilizando el lenguaje python para la creación de una rutina de baja complejidad. Codificar una rutina de baja complejidad en lenguaje python en el	- Reconoce el concepto de variable identificando los tipos de datos fundamentales de acuerdo al lenguaje python. - Elabora expresiones	- Entrada de datos en consola. - Impresión en consola. - Variables. - Tipos de dato fundamentales: - entero - decimal - cadena de caracteres - booleano	2 horas

	entorno de visual studio code.	aritméticas para realizar cálculos de acuerdo al lenguaje python. • Utiliza la entrada y salida estándar de python para el intercambio de datos con la rutina. • Ejecuta un script con instrucciones python desde la consola.	• Expresiones aritméticas. • Conversiones de tipo. • Creación y ejecución de un script python.	
			Desafío - Sistema de gestión de ventas.	2 horas
Sentencias condicionales	Codificar una rutina utilizando estructuras condicionales y expresiones booleanas para resolver un problema de baja complejidad de acuerdo al lenguaje python.	• Identifica los tipos de estructura condicional, así como su utilidad en la confección de una rutina de código. • Elabora expresiones booleanas utilizando operadores booleanos y de comparación para resolver un problema de baja complejidad. • Utiliza estructuras de control de flujo de acuerdo a la sintaxis del lenguaje python para resolver el problema planteado.	• Qué es una sentencia condicional y por qué se necesitan. ○ La sentencia if. ○ La sentencia if-else. ○ La sentencia if-elif-else. • Operadores booleanos: ○ and ○ or ○ negación • Operadores de comparación: ○ Mayor que, mayor o igual que. ○ Menor que, menor o igual que. ○ Igual que. ○ Distinto que. • Paréntesis y expresiones booleanas.	2 horas

Funciones y módulos	Codificar una rutina utilizando funciones predefinidas, funciones personalizadas o de un módulo para resolver un problema de baja complejidad de acuerdo al lenguaje python.	- Reconoce las principales funciones predefinidas disponibles en el lenguaje python. - Elabora funciones personalizadas con parámetros y retorno de acuerdo a la sintaxis del lenguaje python para resolver el problema planteado. - Utiliza funciones pertenecientes a un módulo para resolver un problema de baja complejidad.	- Qué es una función y para qué sirven. - Funciones predefinidas de python. - Funciones personalizadas: - Definición de una función personalizada. - Parámetros de una función y retorno. - Utilización de una función personalizada. - Qué es un módulo y para qué sirven. - La librería standard de python. - Importación de módulos. - El módulo math. - El módulo stat.	2 horas
			Desafío - Calculadora de calificaciones	2 horas
Estructuras de datos en python	Utilizar estructuras de datos para resolver un problema según lenguaje Python.	- Distingue las características de las estructuras de datos de tipo colección que provee el lenguaje python para resolver un problema. - Selecciona las estructuras de datos adecuadas en Python para resolver un problema de baja complejidad. - Utiliza operaciones básicas de listas y	- Qué es una estructura de datos y por qué se necesitan. - Listas. - Características de una lista. - Crear una lista. - Agregar elementos. - Rescatar un elemento. - Rescatar un rango de elementos. - Listas y cadenas de caracteres. - Listas anidadas y matrices. - Diccionarios. - Características de un diccionario. - Crear un diccionario. - Agregar elementos. - Rescatar elementos.	2 horas

		diccionarios en una rutina de código para resolver un problema de baja complejidad.	• Diccionarios anidados. • Tuplas. • Características de una tupla. • Creación de una tupla. • Rescatar elementos. • Empaquetado y desempaquetado de tuplas. • Sets. • Características de un set. • Creación de un set. • Operaciones de conjunto con sets. • Compresión de listas, diccionarios y sets (contenido opcional).	
Sentencias iterativas	Codificar una rutina utilizando sentencias iterativas para resolver un problema de baja complejidad en Python.	• Distingue las características de las sentencias iterativas while y for en la resolución de un problema. • Utiliza la sentencia for para iterar estructuras de datos de tipo colección de acuerdo al lenguaje python. • Codificar una rutina que resuelve un problema de baja complejidad utilizando sentencias iterativas de acuerdo al lenguaje python.	• Qué es una sentencia iterativa y por qué se necesitan. • La sentencia while. • La sentencia for. • Iterando listas de elementos. • Iterando diccionarios de elementos. • La función range. • Iterando con la función range.	2 horas

Conceptos básicos de orientación a objeto en python	Codificar una rutina utilizando clases provistas para la resolución de un problema simple de acuerdo al paradigma de orientación a objetos en el entorno python.	• Explica los elementos fundamentales del paradigma de orientación a objetos en la programación. • Distingue la diferencia entre una clase y un objeto, en el contexto de la programación orientada a objetos. • Implementa una clase utilizando un método inicializador con parámetros y métodos personalizados para resolver un problema de bajísima complejidad. • Utiliza clases y métodos existentes en librerías para la resolución de un problema de baja complejidad.	• En qué consiste el paradigma de orientación a objetos en la programación. • Principios básicos de la programación orientada a objetos: ○ Clases y objetos. ○ Abstracción y ocultamiento. ○ Estado y atributos. ○ Comportamiento y métodos. ○ Constructores e instanciación. • El objeto string y sus métodos principales.	2 horas
			Prueba - Sistema de Gestión de Torneo de Videojuegos Retro	3 horas